



جلسه فنی تیم ۶۰۰ دقیقه

Agentic Coding

از نیازمندی تا تست

از Business Requirement تا Test با Rule، Spec و Skill

Cursor · Claude Code · GitHub Copilot · Codex · Kiro · Spec Kit

Agenda

Core Concepts .ᳵ

LLM, Agent, Context, Harness, Spec / Rule / Skill

?Why now .ᳵ

Data, tool evolution, productivity paradox

Team A to Z .ᳶ

Business ← PO ← Tech Lead ← Developer ← Tester

Spec-Driven Development .᳷

Martin Fowler site / Böckeler + how to write Specs

Practice & FAQ .᳸

How to talk to agents, anti-patterns, roadmap

Tools Comparison .᳹

Which tool for which job, folder structure

Productivity Paradox

مطالعه کنترل شده METR

-19%

توسعه دهندگان باتجربه با AI کندتر شدند، در حالی که فکر می کردند ۲۰٪ سریع ترند.

تسک های واقعی در ریپوهای بزرگ متن باز

گزارش DORA 2025 (گوگل)

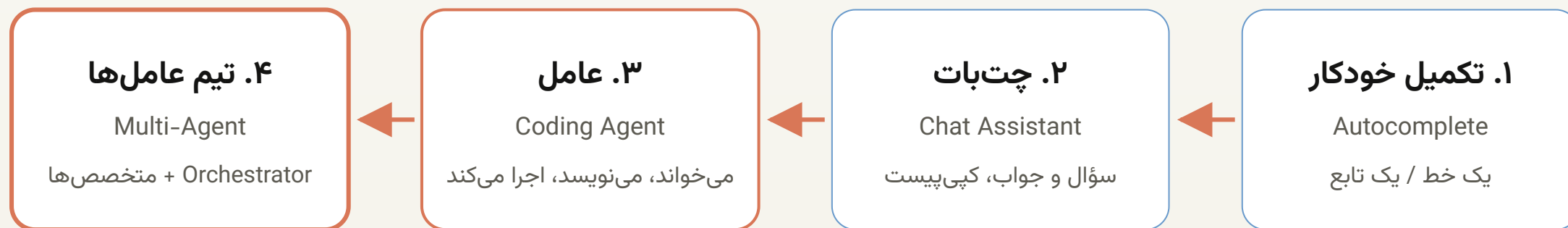
90%

از توسعه دهندگان از AI استفاده می کنند؛ بیش از ۸۰٪ احساس بهره وری بیشتر دارند.

سرعت تحویل ↑ اما پایداری تحویل ↓

«AI یک تقویت کننده است: نقاط قوت و ضعف سازمان را بزرگ تر می کند.»، DORA 2025

From Autocomplete to Agent Teams



نقش انسان: از «تایپ کننده» به «طراح و ناظر حلقه»

02

Core Concepts

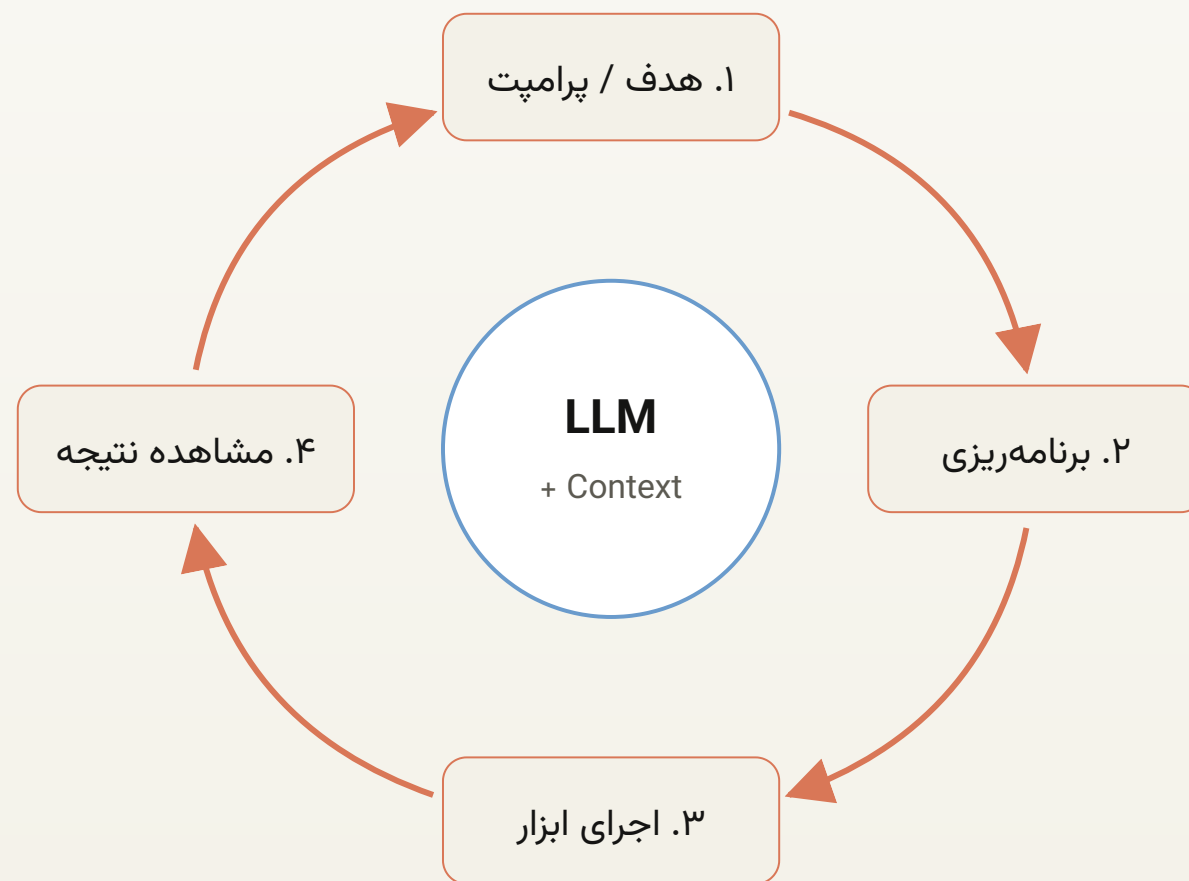
LLM، Agent، Context Engineering، Harness و چهار artifact اصلی

LLM vs Coding Assistant vs Agent

عامل کدنویسی (Agent)	دستیار کدنویسی AI	LLM (مدل زبانی)	
مدل + ابزارها + حلقه تصمیم	مدل + UI داخل IDE	مغز: پیش‌بینی متن/توکن	چیست؟
Cursor Agent, Claude Code, Codex, Kiro	Copilot completions, Chat	Claude, GPT, Gemini	مثال
کل ریپو، ترمینال، تست، مرورگر، MCP	فایل باز / انتخاب‌شده	فقط متن ورودی	دسترسی
خود عامل؛ شما بازبینی می‌کنید	شما (کپی/قبول)	شما	چه کسی اجرا می‌کند؟
فیچر چندفایلی، رفع باگ، refactor، تست	تکمیل سریع، سؤال موضعی	ایده، توضیح، تحلیل	بهترین برای

Agent = Model + Harness - یعنی «مدل» فقط بخشی از ماجراست.

Agent Loop



- ابزارها: خواندن/جستجوی فایل، ویرایش، ترمینال، تست، مرورگر، MCP
- مشاهده: خروجی build، خطای تست، lint
- حلقه تا رسیدن به هدف یا توقف تکرار می‌شود
- بدون معیار «موفقیت» قابل اندازه‌گیری، عامل خودش به خودش نمره می‌دهد!

?What is Context



Context = همه اطلاعاتی که مدل همین لحظه می‌تواند ببیند و بر اساس آن‌ها جواب بدهد.

- پنجره محدود است (Context Window): مثل میز کار؛ جا محدود است.
- اگر میز پر شود، چیزهای قدیمی یا کم‌اهمیت جا می‌افتند.
- هر چیز بی‌ربط روی میز = کیفیت پایین‌تر و هزینه بیشتر.
- پس کار ما: انتخاب هوشمندانه اینکه چه چیزی روی میز بماند.

Context Engineering

«Context engineering یعنی گزینش آنچه مدل می‌بیند، تا نتیجه بهتری بگیرید.»

- Bharani Subramaniam، نقل‌شده در مقاله Birgitta Böckeler، martinoflower.com (فوریه ۲۰۲۶)

Specs

قرارداد فیچر فعلی

Skills

رویه‌های قابل بارگذاری در صورت نیاز

Rules

دستورالعمل‌های دائمی پروژه

خود کدبیس

کد تمیز = بهترین context

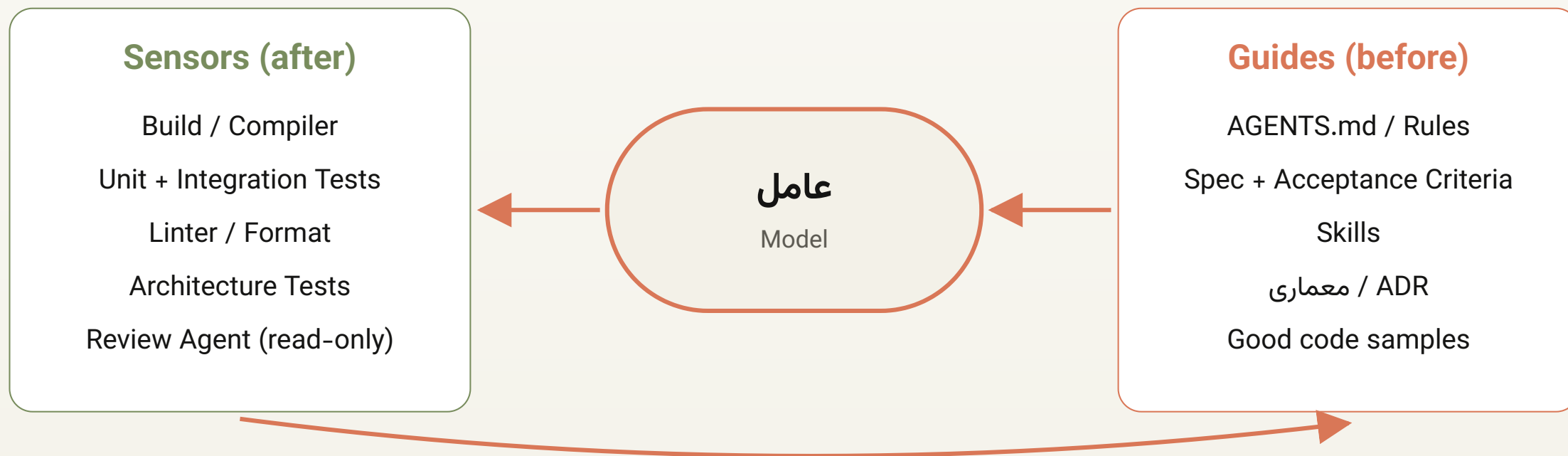
Subagents

context جدا برای تحقیق و بازبینی

MCP

اتصال به DB، Jira، مستندات، مرورگر

Harness: Guides & Sensors



بازخورد: اگر نتیجه بد بود، Harness را اصلاح کن، نه فقط خروجی را

Four Artifacts: Spec / Rule / Skill / Doc

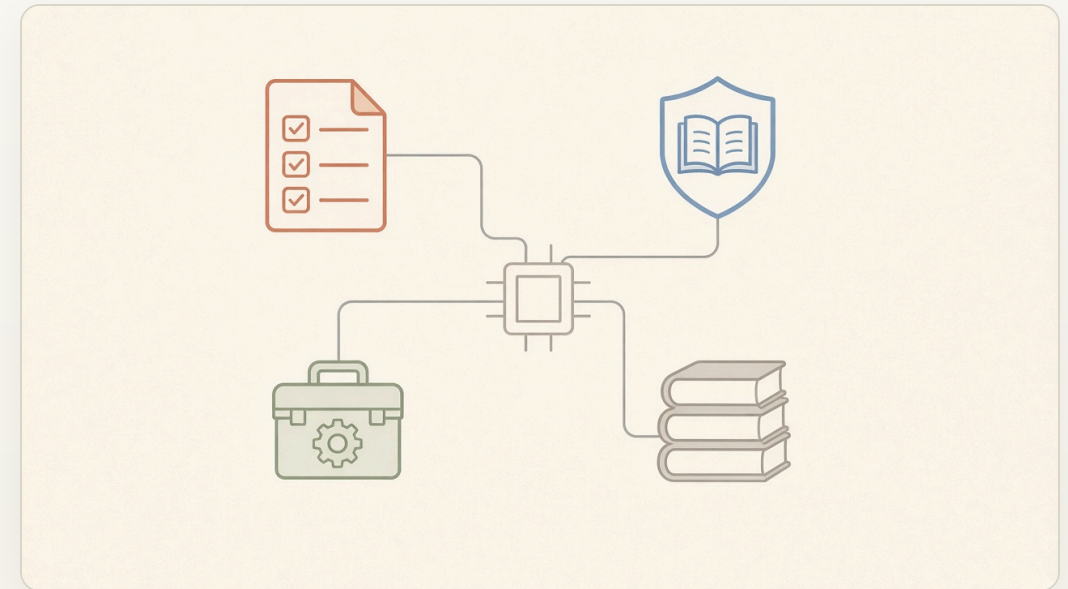
= این بار چه بسازیم **Spec**

= همیشه اینجا چطور کار می‌کنیم **Rule**

= این نوع کار را چطور انجام بدهیم **Skill**

= کسب‌وکار چه معنایی دارد **Business Doc**

اشتباه شماره ۱ تیم‌ها: کپی کردن کل BRD داخل فایل Rule.



? **Spec** What is a

مثال جمله Spec

«Given قسطی با بیش از ۳۰ روز تأخیر، When لیست پیگیری درخواست شود،
Then آن قسط با برچسب Overdue نمایش داده شود.»
قابل تست ✓ ۰ مختص این فیچر ✓

توصیف یک فیچر یا تغییر مشخص: هدف، داستان کاربر، Acceptance Criteria، خارج از محدوده، لبه‌ها.

- **عمر:** برای یک فیچر/تسک (در حالت spec-anchored نگه داشته می‌شود)
- **کی بارگذاری می‌شود:** فقط وقتی روی همان فیچر کار می‌شود
- **نویسنده:** PO + Tech Lead
- **مصرف‌کننده:** عامل توسعه، Developer، Tester
- **محل:** specs/001-feature/spec.md

? Rule What is a

مثال Rule

- Domain must not reference Infrastructure.
- Money fields use decimal, never double.
- Never hardcode secrets.
- Run: dotnet build && dotnet test

دائمی ✓ · کوتاه و دستوری ✓

سیاست **دائمی** پروژه که عامل در تقریباً هر ویرایش مرتبط باید رعایت کند.

- مرزهای معماری (Clean Architecture)
- نام‌گذاری و ساختار پوشه
- حداقل‌های تست، امنیت، دستورهای build
- محل: AGENTS.md، .cursor/rules/*.mdc، CLAUDE.md، .github/copilot-instructions.md
- مالک: Tech Lead، مثل کد review شود

۱. کشف

name + description (~100 توکن)

۲. فعال سازی

کل SKILL.md (> ۵۰۰۰ توکن)

۳. اجرا

references / scripts فقط در صورت نیاز

Progressive Disclosure

یک رویه قابل استفاده مجدد که عامل فقط وقتی لازم است بارگذاری می کند. استاندارد باز (اصالتاً از Anthropic) که Claude Code، Copilot و Cursor، Codex پشتیبانی می کنند.

```
.agents/skills/dotnet-cqrs-feature/  
  SKILL.md           # name + description + steps  
  references/         # optional docs  
  scripts/           # optional helpers  
  assets/            # optional templates
```

Business rule: «اقساط معوق بیش از ۳۰ روز باید پیگیری شوند»

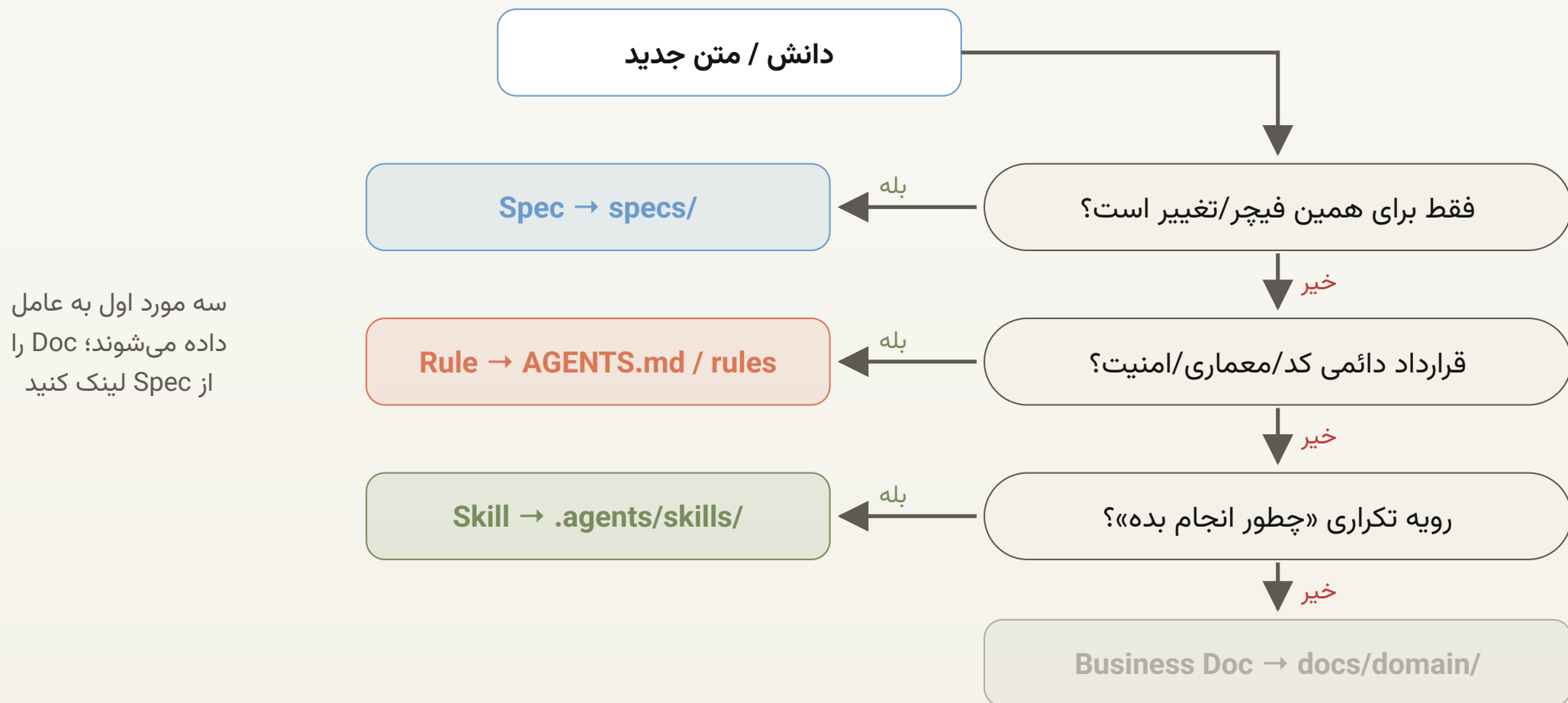
- ✓ در مستند دامنه (حقیقت برای انسان)
- ✓ در Spec فیچر مربوطه، به شکل Acceptance Criteria
- ✓ اختیاری: در references یک Skill اگر مکرر لازم است
- ✗ در cursor/rules، مگر قرارداد دائمی کدنویسی باشد

مستندات کسب و کار = دانش دامنه برای انسان‌ها (و گاهی عامل):
Glossary، روایت فرایند، BRD، نقشه فرایند.

به طور خودکار بارگذاری نمی‌شود؛ از Spec به آن لینک بدهید یا خلاصه‌اش را در Skill/references بگذارید.

محل پیشنهادی: docs/domain/glossary.md،
docs/domain/installments.md

؟Decision Tree: Spec / Rule / Skill / Doc



Skills vs Rules

Skills	Rules	
آموزش یک قابلیت یا گردش کار تخصصی	استاندارد و قرارداد کدنویسی	هدف
در صورت نیاز (تدریجی)	همیشه یا با glob (کل متن)	بارگذاری
دستورالعمل + اسکریپت + مثال + منابع	فقط دستورالعمل	محتوا
می‌تواند بلندتر باشد	کوتاه (هزینه context دائمی)	طول مناسب
استاندارد باز واحد (agentskills.io)	فرمت‌های مختلف به ازای هر ابزار	قابلیت حمل
«مراحل افزودن فیچر CQRS» / «ساخت migration»	«Domain به Infrastructure وابسته نباشد»	مثال
اگر Skill همیشه لازم است و کوتاه ← Rule کن	اگر Rule شبیه آموزش گام‌به‌گام شد ← Skill کن	سیگنال جابه‌جایی

MCP, Subagent & Hook

Hook

اسکریپت قطعی در رویدادهای عامل (قبل/بعد ابزار، پایان).

«Rule توصیه است؛ Hook تضمین است.»

Subagent

عامل کمکی با **context جدا** و ابزار محدود؛ فقط خلاصه برمی‌گرداند.

تحقیق کدبیس، بازبینی read-only، تست

MCP

Model Context Protocol، پروتکل استاندارد اتصال عامل به سیستم‌های بیرونی.

Jira، GitLab، دیتابیس، مستندات، مرورگر

مثال: بعد از هر ویرایش فایل cs. ← Hook اجرای `dotnet build`؛ قبل از `subagent` ← `commit` بازبین.

03

Spec-Driven Development

Spec-Driven Development، دیدگاه martinfowler.com و نحوه نوشتن Rule، Spec، و Skill

Why Spec? Vibe Coding vs Structured

Spec-Driven

- نیازمندی ← Tasks ← Plan ← Spec ← کد
- همراستایی PO و فنی قبل از مصرف توکن
- Acceptance Criteria = حسگر عامل
- Spec در ریپو می ماند و نسخه بندی می شود
- بازبینی در برابر Spec، نه حافظه

Vibe Coding

- پرامپت ← کد ← امید!
- عامل «چیز اشتباهی» را درست می سازد
- هیچ Acceptance Criteria ی برای بازبینی نیست
- دانش در تاریخچه چت گم می شود
- برای نمونه اولیه خوب؛ برای محصول پرریسک

DX / Laura Tacho: مشکل اصلی تیم ها «همراستایی» است، SDD تلاشی برای حل همین است.

۱. Spec-first

Spec خوب اول نوشته می‌شود و برای همین تست استفاده می‌شود؛ بعد از تست ممکن است کنار گذاشته شود.

۲. Spec-anchored

Spec بعد از اتمام تست نگه داشته می‌شود و برای تکامل و نگهداری همان فیچر استفاده می‌شود.

۳. Spec-as-source

فقط Spec توسط انسان ویرایش می‌شود؛ کد تولیدی است و دستی ویرایش نمی‌شود. (هنوز آزمایشی)



How SDD Helps the Team

کاهش دوباره کاری

کمتر «عامل چیز اشتباه ساخت»

Quality gate

Acceptance Criteria ← تست ← حسگر عامل

همراستایی

PO و Tech Lead قبل از کد توافق می کنند

رشد جونیورها

Spec = بریف، Skill = روش انجام

بازبینی بهتر

Reviewer در برابر Spec چک می کند

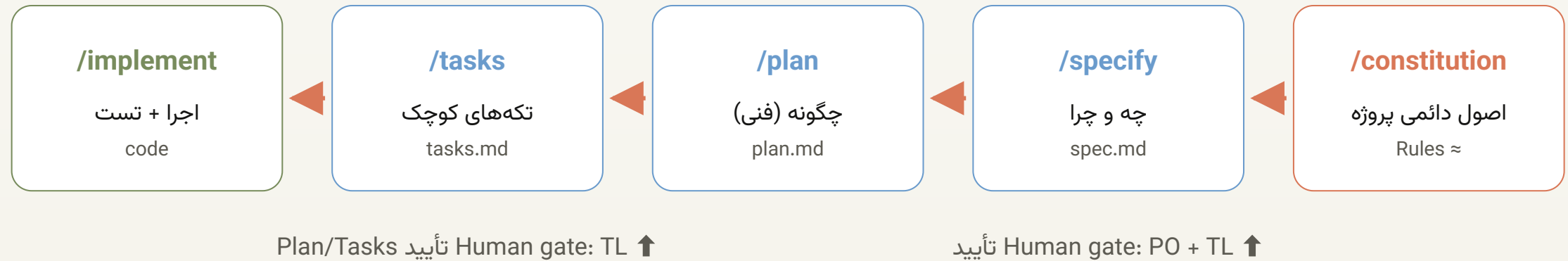
تحويل شفاف

Tasks ← Plan ← Spec ← BRD ← کد ← تست

Kiro / Spec Kit / Tessl

Tessl	Spec Kit (GitHub)	Kiro (AWS)	
فریم‌ورک + رجیستری Skill	CLI + slash command در ابزار شما	IDE agentic	نوع
Spec ← کد تولیدی	Constitution ← Specify ← Plan ← Tasks ← Implement	Requirements ← Design ← Tasks	جریان
تنها هدف‌گذاری -as / spec-anchored source	قابل سفارشی‌ترین، عملاً spec-first	سبک، spec-first	سطح (Böckeler)
...، Claude Code، Codex، Gemini	~۳۰ عامل: Copilot، Claude، Cursor، ...	خود Kiro	ابزارهای پشتیبان
آزمایش آینده‌نگر	تیم‌هایی که artifact در ریپو می‌خواهند	تیم‌های AWS، شروع سریع	مناسب برای

نقد رایج کاربران: «Spec Kit گاهی شبیه آبشار (waterfall) برای عامل‌هاست»، Spec را سبک نگه دارید.



نمونه کامل در ریپو: `samples/specs/001-installment-followup/` (spec.md, plan.md, tasks.md)

چک لیست کیفیت

- ☐ هر معیار با تست یا دمو قابل اثبات است
- ☐ معماری مفصل اینجا نیست (برود در Plan / ADR)
- ☐ اندازه یک PR یا اپیک کوتاه
- ☐ واژه‌ها هم‌خوان با Glossary کسب‌وکار
- ☐ Non-goals روشن (جلوی scope creep عامل)
- ☐ بعد از merge نگه داشته و به‌روز شود

قالب حداقلی

1. هدف (یک پاراگراف)
2. داستان‌های کاربر
3. **Acceptance Criteria** قابل تست (Given/When/Then)
4. خارج از محدوده (Non-goals)
5. قوانین دامنه مرتبط (لینک به docs)
6. لبه‌ها و خطاها
7. وابستگی‌ها
8. Definition of Done

Sample Spec: Installment Follow-up

هدف: فروشنده اقساط معوق را شناسایی و اقدام پیگیری ثبت کند تا وضعیت شفاف شود.

:Acceptance Criteria

1. Given قسط با بیش از ۳۰ روز تأخیر، When لیست درخواست شود، Then با برچسب Overdue بیاید.
2. Given قسط معوق، When پیگیری ثبت شود، Then رکورد با CreatedAt و CreatedBy ذخیره شود.
3. Given پیگیری‌ها، When جزئیات خوانده شود، Then تاریخچه جدید به قدیم برگردد.
4. Given ورودی نامعتبر، When ثبت شود، Then خطای اعتبارسنجی/NotFound و هیچ ذخیره‌ای.

خارج از محدوده: ارسال خودکار پیامک، محاسبه جریمه، ال وب.

Domain rule (از docs): هر اقدام پیگیری باید قابل ممیزی باشد.

Rule How to Write a Good

.cursor/rules/dotnet-clean-architecture.mdc

```
---
description: Clean Architecture boundaries
globs: "**/*.{cs,csproj}"
alwaysApply: false
---
# .NET Clean Architecture
## Do
- Depend inward: Host → Application → Domain
- Business rules in Domain/Application
- Use Result for expected failures
## Don't
- Reference Infrastructure from Domain
- Use DbContext in controllers
- Invent folder layouts without an ADR
```

• ✓ خوب:

- کوتاه، دستوری: Do / Don't
- دائمی، نه مخصوص یک تیکت
- با globs محدود شود
- چند فایل کوچک بهتر از یک فایل غول

• ✗ بد:

- مقاله بلند / آموزش گام به گام (← Skill)
- دستور مخصوص یک فیچر (← Spec)
- کپی (← docs) BRD

`.agents/skills/dotnet-cqrs-feature/SKILL.md`

```

---
name: dotnet-cqrs-feature
description: Add a MediatR CQRS feature (Command/Query,
  Handler, Validator, Controller). Use when the user asks
  to add a new API feature, use-case, command, or query.
---
## Steps
1. Read specs/**/spec.md (acceptance criteria)
2. Application: Command/Query + Handler + Validator
3. Infrastructure: repository methods if missing
4. Host: thin controller → MediatR
5. Tests: happy path + validation failure
6. Verify: dotnet build && dotnet test
## Output checklist
- [ ] No business logic in controller
- [ ] Spec acceptance criteria covered

```

- name: کوچک، با خط تیره، هم‌نام پوشه
- description: چه می‌کند + کی فعال شود (تمام بار trigger)
- بدنه: مراحل شماره‌دار، دستور دقیق، چک‌لیست خروجی
- زیر ~۵۰۰۰ توکن؛ اضافه‌ها در references/
- با یک کار واقعی تیم تست شود
- رویه تکراری باشد، نه فیچر یک‌باره

Pitfalls & Critiques

⚠️ آبخار برای عامل‌ها

Spec سنگین و فرم‌های زیاد = از دست رفتن چابکی. سبک شروع کنید.

⚠️ اصطلاحات هنوز سیال است

Böckeler: حتی درک مشترکی از «Spec» و تفاوتش با instruction file وجود ندارد.

⚠️ Rules داخل BRD

context شلوغ، قوانین مبهم، هزینه توکن بالا، کیفیت پایین.

⚠️ رانش Spec (Drift)

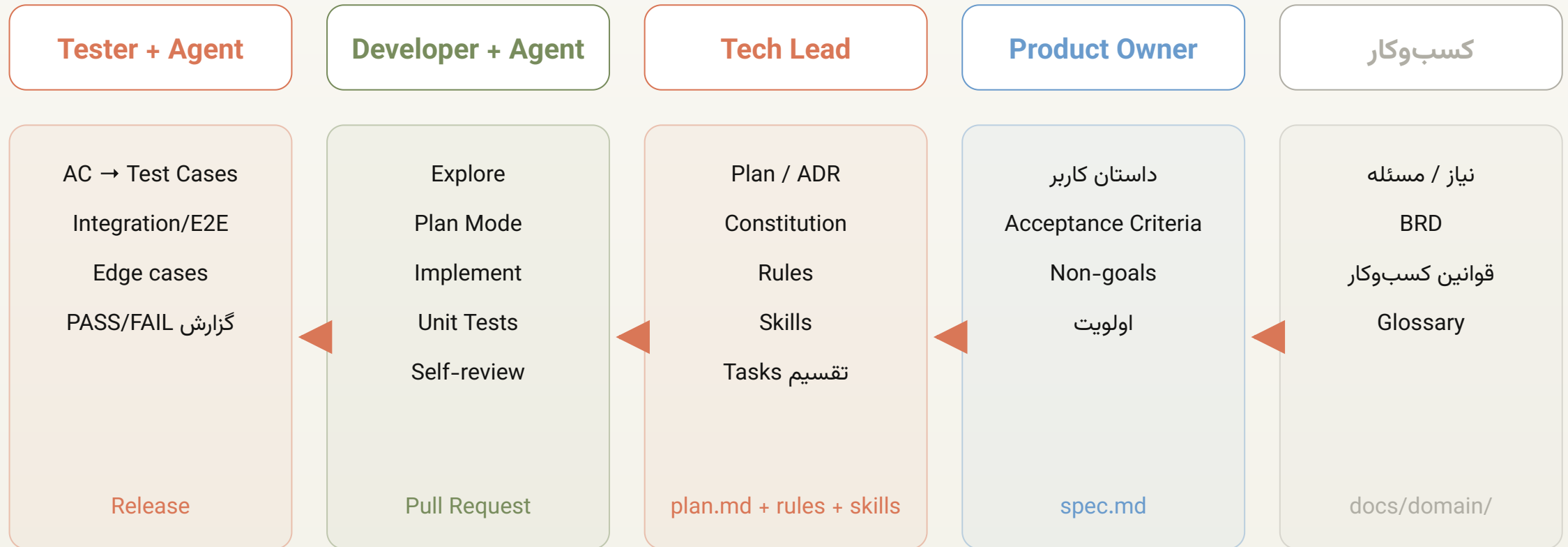
Spec-first بدون به‌روزرسانی ← Spec دروغ می‌گوید. فیچرهای مهم را anchored کنید.

04

Team A to Z

کسب و کار ← Tester ← Developer ← Tech Lead ← PO، هر نقش با عامل‌ها

End-to-End Flow: Requirement → Test



PO & Tech Lead + Agents

Tech Lead

- مالک Rules، Constitution، AGENTS.md
- ساخت و نگهداری Skills تیم
- تأیید Plan و تقسیم Tasks
- طراحی Harness: تست معماری، CI، Hook
- تعیین سطح Autonomy عامل برای هر نوع کار
- خروجی: plan.md + rules + skills

Product Owner

- تبدیل BRD به داستان کاربر با کمک عامل
- درخواست از عامل: «Acceptance Criteria Given/When/Then» و لبه‌ها را پیشنهاد بده»
- اتصال MCP به Jira: ساخت تیکت از Spec
- بررسی Spec از نظر ابهام و تناقض
- خروجی: spec.md تأییدشده

Developer & Tester + Agents

Tester / QA

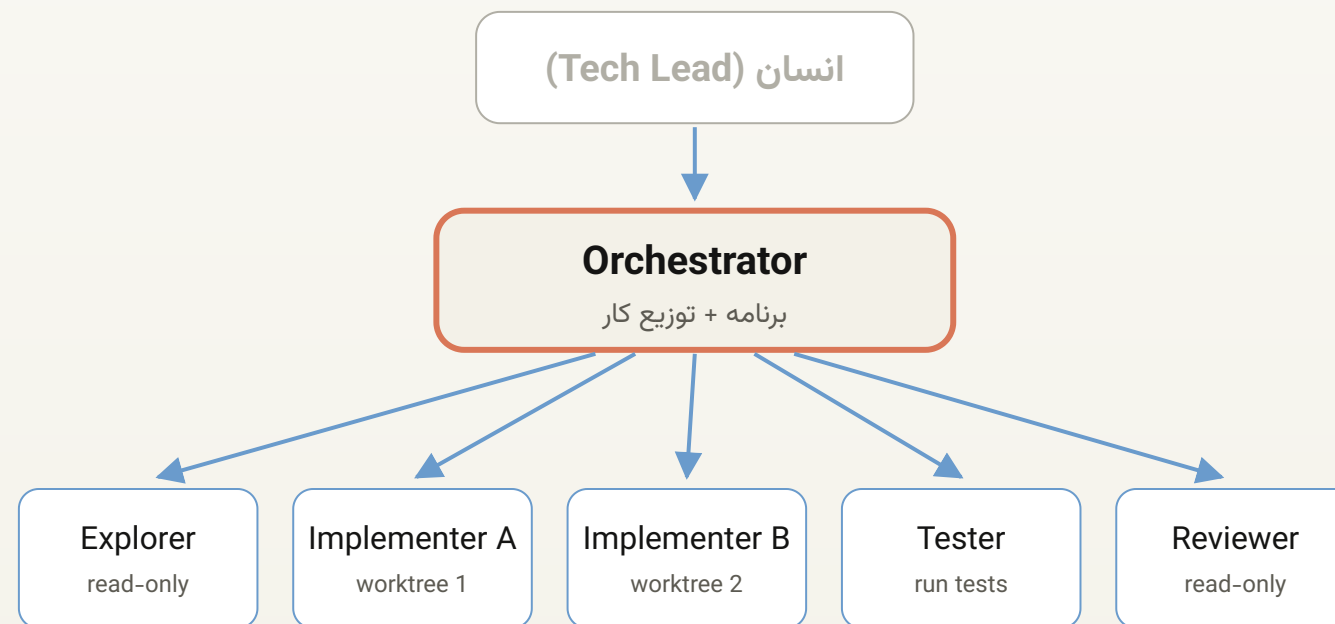
- تبدیل Acceptance Criteria Spec به test case
- تولید تست integration / E2E با عامل
- درخواست: «لبه‌هایی که Spec پوشش نداده را پیدا کن»
- عامل مرورگر برای تست UI
- گزارش شکاف Spec به PO
- خروجی: PASS/FAIL در برابر AC

Developer

- **Explore:** «با subagent بررسی کن follow-up فعلی چطور کار می‌کند»
- **Plan:** Plan Mode، فایل‌ها و ترتیب را تأیید کن
- **Implement:** با dotnet-cqrs-feature Skill
- **Verify:** build + test تا سبز شدن
- **Review:** subagent بازبین read-only قبل از PR
- مالک نهایی کد = انسان

Agent Orchestration

- Orchestrator خودش کد نمی‌نویسد؛ تقسیم می‌کند
- هر متخصص context و ابزار محدود دارد
- کار موازی در **git worktree** جدا
- Explorer و Reviewer فقط خواندنی
- حداکثر ۳ دور اصلاح، بعد ارجاع به انسان



دروازه‌های انسانی: تأیید Plan · تأیید Merge
PR عامل = همان CI و branch protection مثل PR انسان

✓ On the Loop

انسان **Harness** را می‌سازد و بهبود می‌دهد: Spec،
تست، Rule، Skill، Pipeline.
مقیاس‌پذیر

In the Loop

انسان هر خط را بازبینی و دستی اصلاح می‌کند.
گلوگاه می‌شود

Vibe Coding

انسان بیرون از حلقه؛ هر چه عامل ساخت قبول.
برای نمونه‌سازی

وقتی از خروجی راضی نیستید: «درون حلقه» = خروجی را اصلاح کن. «روی حلقه» = چیزی که آن خروجی را تولید کرد اصلاح کن.

Verification Gate & Autonomy

نوع کار	Autonomy
مستندات، تست نویسی ماژول پوشش دار	بالا
ارتقای وابستگی با تست کامل	بالا
فیچر معمولی با Spec	متوسط (تأیید Plan)
احراز هویت، پرداخت، بیمه نامه	پایین (پیشنهاد فقط)
Migration داده	پایین + تأیید صریح

دروازه «انجام شد» (Verification Gate)

1. تست های جدید/مرتبط پاس
 2. کل suite پاس (بدون regression)
 3. Build و lint تمیز
 4. معیارهای پذیرش Spec پوشش داده شده
 5. برای تغییر پریسک: تأیید انسان
- «ارزیابی خود مدل از اتمام، کم اعتمادترین سیگنال است.»

05

Tools Comparison

Cursor · Claude Code · GitHub Copilot · Codex · Kiro · Spec Kit

Tools Matrix (2026)

ابزار	رابط	مدل‌ها	نقطه قوت	مناسب برای
Cursor	IDE کامل + عامل پس‌زمینه	چند مدل	ویرایش تعاملی، Rules با subagent، glob	کار روزانه داخل IDE
Claude Code	ترمینال + IDE	Anthropic	استدلال عمیق، refactor بزرگ، Skill، Hook	کارهای چندفایلی سخت
GitHub Copilot	افزونه IDE + GitHub + CLI	چند مدل	یکپارچگی GitHub، Issue→PR، code review	سازمان‌های GitHub-محور
OpenAI Codex	CLI + ابر	OpenAI	تسک ناهمگام در sandbox ابری	«بفرست و فراموش کن»
Kiro	IDE	چند مدل	Spec-first داخلی	تیم‌های AWS، شروع SDD
Spec Kit	CLI روی ابزار شما	-	قالب SDD برای ~۳۰ عامل	افزودن SDD به ابزار فعلی

بسیاری از تیم‌ها دو ابزار را ترکیب می‌کنند: یکی برای ویرایش روزانه، یکی برای کارهای سنگین واگذارشده.

Where Each Tool Reads Rules & Skills

ابزار	Rules / Instructions	Skills	Subagents
Cursor	.cursor/rules/*.mdc, AGENTS.md, CLAUDE.md	.agents/skills, .cursor/skills	✓
Claude Code	CLAUDE.md (+ @AGENTS.md import)	.claude/skills	.claude/agents/*.md
Copilot	.github/copilot-instructions.md, .github/instructions/*.instructions.md, AGENTS.md	.github/skills, .agents/skills	(agent mode) ✓
Codex	AGENTS.md (nested)	.agents/skills	✓
Kiro	.kiro/steering/*.md	-	-

AGENTS.md = استاندارد بی طرف (بنیاد ۶۰، Linux هزار+ ریپو) · نزدیک ترین فایل به کد ویرایش شده برنده است.

Repo Folder Structure

- ریشه + .cursor/rules **Rule**
- .agents/skills **Skill**
- specs/ **Spec**
- docs/ **Doc**
- همه در git، همه review می‌شوند
- نمونه کامل در پوشه samples/ این ریپو

```
repo/
├── AGENTS.md                # single source of truth (rules)
├── CLAUDE.md                # "@AGENTS.md" + Claude-only notes
├── .github/
│   └── copilot-instructions.md # short, points to AGENTS.md
├── .cursor/rules/
│   ├── dotnet-clean-architecture.mdc # globs: **/*.cs
│   └── testing.mdc                  # globs: **/*Tests*/**
├── .agents/skills/          # cross-tool skills standard
│   ├── dotnet-cqrs-feature/SKILL.md
│   └── ef-migration/SKILL.md
├── .claude/agents/
│   └── code-reviewer.md       # read-only subagent
├── specs/
│   └── 001-installment-followup/
│       ├── spec.md
│       ├── plan.md
│       └── tasks.md
├── docs/
│   ├── domain/glossary.md    # business knowledge
│   └── adr/                  # architecture decisions
└── src/ tests/
```


?Which Tool for Which Job

ویرایش روزانه، فیچر کوچک

Cursor Agent / Copilot Agent mode

سؤال سریع، توضیح کد

چت با LLM یا Ask mode در IDE

کار موازی ناهمگام

Codex cloud / Cursor background agents / Copilot coding agent

Refactor بزرگ، باگ پیچیده

Plan Mode + subagent با Claude Code / Cursor

بازبینی PR

Copilot code review / Bugbot / subagent read-only

فیچر جدید با ذی نفعان متعدد

Kiro (SDD) یا Spec Kit

مهمتر از ابزار: Harness شما (Rules، Specs، Skills)، اینها بین ابزارها قابل انتقال اند.

06

Practice & FAQ

چطور با عامل صحبت کنیم، ضدالگوها، نقشه راه، FAQ

Prompt Anatomy for Agents

۱. هدف

چه نتیجه‌ای می‌خواهی؟ (نه فقط «درستش کن»)

۲. Context

Spec، فایل‌ها، کد مشابه موجود، تیکت

۳. محدودیت‌ها

چه چیزی را تغییر نده، Non-goals، الگو

۴. تأیید

چطور بفهمیم درست است؟ تست، AC، build

۵. خروجی

Plan؟ PR؟ خلاصه؟ قالب مشخص

۶. حالت

اول Plan Mode، بعد اجرا

قاعده Anthropic: اگر diff را در یک جمله می‌توانی توصیف کنی، Plan لازم نیست؛ وگرنه اول Plan.

Bad Prompt vs Good Prompt

✗ بد

یه API برای پیگیری اقساط بنویس.

بدون Spec، بدون محدودیت، بدون معیار تأیید ← عامل حدس می‌زند.

✓ خوب

فیچر specs/001-installment-followup/spec.md را پیاده کن.

۱. اول با subagent بررسی کن InstallmentService فعلی چطور کار می‌کند.

۲. در Plan Mode فایل‌ها و ترتیب را پیشنهاد بده و صبر کن تا تأیید کنم.

۳. از dotnet-cqrs-feature Skill پیروی کن.

۴. به Domain و جدول‌های موجود دست نزن؛ ارسال پیامک خارج از محدوده است.

۵. هر ۱ Acceptance Criteria تا ۴ باید یک unit test داشته باشد؛ dotnet test سبز.

۶. در پایان خلاصه تغییرات + نگاشت AC به تست بده.

Anti-patterns

✗ پذیرش بدون خواندن

مالک کد شماييد؛ PR عامل = همان review.

✗ يك چت بي پايان

context پر و آلوده مي شود؛ براي هر تسك جلسه جديد.

✗ «درستش كن» بدون تست

عامل به خودش نمره مي دهد.

✗ AGENTS.md هزار خطي

هر خط هزينه دائمي context دارد.

✗ اصلاح دستي خطاي تكراري

به جايش Rule/Skill/تست اضافه كن (روي حلقه).

✗ Secret در پرامپت يا Rule

رمز و توكن هرگز در متن عامل.

Roadmap 90 / 60 / 30

۳۰ روز اول

- AGENTS.md برای ۲ سرویس
- Rule ۲ (معماری، تست)
- آموزش Plan Mode
- اندازه‌گیری پایه: lead time، باگ

۶۰ روز

- Skill ۳ مشترک (.NET CQRS، migration، تست)
- اولین فیچر با Spec کامل
- subagent بازبین در PR
- Hook برای build/test

۹۰ روز

- SDD برای همه فیچرهای جدید
- Spec-anchored برای ماژول‌های اصلی
- کتابخانه Skill تیمی
- بازنگری داده‌ها و تنظیم Autonomy

نقش‌ها: Champion (پیشگام) · Tech Lead (مالک Harness) · Manager (حمایت و اندازه‌گیری)

FAQ (1)

Business rule Rule است یا Skill؟

معمولاً هیچ کدام: مستند دامنه + Spec. Rule Acceptance Criteria فقط قرارداد کدنویسی.

آیا AI جای توسعه دهنده را می گیرد؟

نه؛ نقش را تغییر می دهد: طراحی، Spec، بازبینی، Harness. مالکیت و مسئولیت با انسان است.

Spec-first یا Spec-anchored؟

شروع با spec-first؛ برای مازول های مهم و پرتغییر spec-anchored.

BRD را داخل AGENTS.md بگذاریم؟

نه. BRD در docs/؛ خلاصه قابل تست در Spec.

FAQ (2)

توهم (Hallucination)؟

با context درست، تست قطعی، و بازبینی کاهش می‌یابد، حذف نمی‌شود.

امنیت کد شرکت؟

Privacy mode، عدم ارسال secret، بررسی سیاست داده هر ابزار، محدودیت دسترسی MCP و ترمینال.

کدام مدل؟

مدل قوی برای Plan و کار سخت؛ مدل سریع/ارزان برای کار تکراری. Harness مهم‌تر از مدل است.

جونیورها چطور یاد بگیرند؟

از عامل «توضیح» بخواهند، Plan را بخوانند، و کد را خودشان review کنند؛ نه فقط accept.

References

- Böckeler. Understanding Spec-Driven-Development: Kiro, spec-kit, and Tessel. martinfowler.com/articles/exploring-gen-ai/sdd-3-tools.html
- Böckeler. Context Engineering for Coding Agents. martinfowler.com/articles/exploring-gen-ai/context-engineering-coding-agents.html
- Böckeler. Harness engineering for coding agent users. martinfowler.com/articles/harness-engineering.html
- Kief Morris. Humans and Agents in Software Engineering Loops. martinfowler.com/articles/exploring-gen-ai/humans-and-agents.html
- Thoughtworks Podcast. What is spec-driven development? (Böckeler & Laura Tacho)
- Agent Skills specification. agentskills.io · github.com/agentskills/agentskills
- AGENTS.md. agents.md
- Cursor docs. Rules & Skills. cursor.com/docs
- Anthropic. Best practices for Claude Code. code.claude.com/docs/en/best-practices
- VS Code. Agent Skills. code.visualstudio.com/docs/agent-customization/agent-skills
- GitHub Spec Kit. github.com/github/spec-kit
- DORA 2025. State of AI-assisted Software Development. dora.dev/research/2025
- METR. Impact of Early-2025 AI on Experienced OSS Developer Productivity. metr.org



Thanks

Q & A

همه فایل‌ها، نمونه‌ها و قالب‌ها روی گیت‌هاب:

github.com/mohammadrezatahmasebi/ai-agentic-meeting

[/slides.pdf](#) · [handbook.pdf](#) · [samples/](#) · [templates](#)